Team Elha2

Elha2: A Food Saving App for Egypt

Software Design Document

Version 1.0

Name(s): Malak Dessouky
Yassin Shamaa
Ali Montasser
Farida Fayek
Ibrahim Meligy

Course: Software Engineering, Fall 2026

Lab Section: –

Workstation: –

Date: 10/10/2026

American University in Cairo

Structure: SDD template summarised from IEEE Std 1016 (Recommended Practice for Software Design Descriptions).

Contents

Revision History **iii**

1 Introduction **1**
1.1 Purpose . . . 1
1.2 Scope . . . 1
1.3 Overview . . . 1
1.4 Reference Material . . . 2
1.5 Definitions and Acronyms . . . 2

2 System Overview **4**
2.1 Functionality . . . 4
2.2 Context . . . 4
2.3 Design Summary . . . 5
2.4 Design Stakeholders and Concerns . . . 5

3 System Architecture **6**
3.1 Architectural Design . . . 6
3.2 Decomposition Description . . . 10
3.3 Design Rationale . . . 16

4 Data Design **21**
4.1 Data Description . . . 21
4.2 Data Dictionary . . . 24

5 Component Design **28**
5.1 AI Gateway: Run an AI Task . . . 28
5.2 Pantry: Scan and Confirm a Receipt . . . 28
5.3 Expiry: Estimate an Expiry Date . . . 29
5.4 Pantry: Choose the Weekly Check Cards . . . 29
5.5 Recipes: Suggest Recipes for At-Risk Food . . . 30
5.6 Marketplace and Orders: Reserve a Bag . . . 30
5.7 Orders: Collect an Order . . . 31
5.8 Impact Ledger: Record a Verified Action . . . 31
5.9 Donations: Route Unsold Food to an NGO . . . 32
5.10 Notifications: Send a Notification . . . 32
5.11 Mobile App: Synchronise Offline Changes . . . 33

6 Human Interface Design **34**

6.1 Overview of User Interface 34
6.2 Screen Images 35
6.3 Screen Objects and Actions 35

7 Requirements Matrix **38**
7.1 Components to Requirements 38
7.2 Functional Requirements to Components and Data 39

8 Appendices **42**
Appendix A: Deployment Design 42
Appendix B: Design Change Procedure 42
Appendix C: Declaration of AI Use 43

Revision History

Every sprint adds a row here. Text added or changed in a version is printed in blue with the version in the margin, and removed text is struck through.

Name	Date	Reason For Changes	Version
Elha2 team	10/10/2026	Milestone 1 baseline in the course SDD template: high-level architecture with UML component diagrams, architectural and design patterns, quality attributes, class and sequence diagrams, data dictionary, component pseudocode, low-fidelity wireframes and requirements matrix	1.0

1. Introduction

1.1 Purpose

This Software Design Document (SDD) describes the architecture and system design of Elha2, release 1.0: the subsystems and components it is built from, how they collaborate, the data they store, the algorithms of the main operations, and the user interface. It translates the requirements in the Elha2 Software Requirements Specification (SRS) [1] into a representation that programmers can implement.

Its intended audience is the five-member development team, who use it as the primary reference for writing code; our instructor and teaching assistants, who assess the design at each milestone; and future maintainers. The SDD is updated at the end of every sprint together with the SRS, and every change is recorded in the revision history.

1.2 Scope

Elha2 is an Arabic-first, AI-assisted mobile app and web portal that reduces food waste in Egypt at three points: *Prevent* (a smart household pantry that fills itself from receipts, photos and voice notes and warns before food spoils), *Sell* (businesses list surplus at a discount for pickup) and *Donate* (unsold food goes to NGOs automatically, and households can donate sealed items). Every verified action is measured in kilograms, Egyptian pounds, CO_2e and meals.

The goals are to cut household food waste, to make household tracking effortless, to recover business surplus as affordable food, to redistribute food to people in need, and to measure impact credibly, while keeping food, users and data safe (SRS Section 1.4, Product Scope). The benefits are lower grocery bills, recovered revenue for businesses, a steady flow of safe food for NGOs, and credible data for SDG 12.3 reporting.

This SDD covers the whole system specified in the SRS, so that the architecture does not need to change as later features are added. The *preliminary design* (architecture and data architecture, Chapters 3 and 4) covers all 14 features. The *detailed design* (component algorithms and screens, Chapters 5 and 6) is given for the 39 stories we implement during the course (SRS Section 6.6) and is extended sprint by sprint.

1.3 Overview

Chapter 2 gives a general description of the system's functionality, context and design. Chapter 3 presents the architecture: the subsystems and their responsibilities (Section 3.1), their object-oriented decomposition with component, sequence and interface descriptions (Section 3.2), and the rationale for the design (Section 3.3). Chapter 4 describes how data is stored and organised, with

UML class diagrams and a data dictionary. Chapter 5 gives pseudocode for the main operations of each component. Chapter 6 describes the user interface with low-fidelity wireframes. Chapter 7 traces every component and every SRS requirement to each other. The appendices hold the deployment design and the declaration of AI use.

1.4 Reference Material

[1] Elha2 team, *Elha2 Software Requirements Specification, Version 1.0*. American University in Cairo, 2026.

[2] Course instructor, *Software Engineering Project: Project Description and Milestones*. American University in Cairo, Fall 2026.

[3] IEEE, *IEEE Std 1016-2009: IEEE Standard for Information Technology – Systems Design – Software Design Descriptions*. New York: IEEE, 2009.

[4] Object Management Group, *Unified Modeling Language (UML), Version 2.5.1*, 2017.

[5] E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[6] L. Bass, P. Clements and R. Kazman, *Software Architecture in Practice*, 4th ed. Addison-Wesley, 2021.

[7] M. Dessouky, Y. Shamaa, A. Montasser, F. Fayek and I. Meligy, *Elha2: A Food Saving App for Egypt – Phase 0: Ideation*. American University in Cairo, 2026.

[8] Google, *Material Design 3 Guidelines*, m3.material.io.

1.5 Definitions and Acronyms

Terms used in the requirements (for example LLM, OCR, PDPL, verified action) are defined in SRS Appendix A. The terms below are specific to this design.

Term	Definition
Adapter	A design pattern: a class that wraps an external service behind an interface the rest of the code owns.
AI gateway	The single backend service through which every call to a hosted AI model passes.
API	Application programming interface; here, mostly the REST API between the clients and the backend.
Component	A replaceable part of the system that provides and requires interfaces (UML component).
DTO	Data transfer object: the JSON shape of a request or response.
Idempotency key	A unique key sent with a request so that a retry does not repeat its effect.
Impact ledger	The append-only table of verified impact records and the service that writes it.

Term	Definition
JWT	JSON Web Token; Firebase ID tokens are JWTs.
Modular monolith	One deployable backend divided into modules that talk only through declared interfaces.
NestJS	A TypeScript framework for Node.js server applications, organised in modules.
ORM	Object-relational mapper: maps TypeScript classes to PostgreSQL tables (planned: Prisma).
PDL	Procedural description language (pseudocode), used in Chapter 5.
pg-boss	A job queue for Node.js that stores its jobs in PostgreSQL.
Provided interface	An interface a component offers to others; drawn as a ball ("lollipop").
Riverpod	A state-management library for Flutter.
Subsystem	A group of components that is designed and deployed together.
Usage dependency	A dashed arrow from a component to an interface it requires.
WebSocket	A persistent two-way connection used for live updates.

2. System Overview

2.1 Functionality

Elha2 serves six kinds of user through two clients. Households use the **mobile app** to keep a smart pantry (filled from receipts, barcodes, photos and voice notes and checked once a week), receive expiry reminders, cook suggested Egyptian recipes, buy discounted surplus, donate sealed food to NGOs and see their impact. Businesses, NGOs, administrators and government analysts use the **web portal** to list surplus and confirm pickups, receive donation offers, run the platform, and export anonymised data. The functions are grouped into 14 features (E1 to E13), specified as 100 user stories in SRS Chapter 4.

2.2 Context

Elha2 is a new, self-contained product. Figure 2.1 shows its context: the people and organisations it serves and the external systems it depends on for login, push notifications, maps, payments, AI models, product data, weather and grocery emails. Every external system is reached through an adapter or the AI gateway, so it can be replaced without changing the rest of the code.

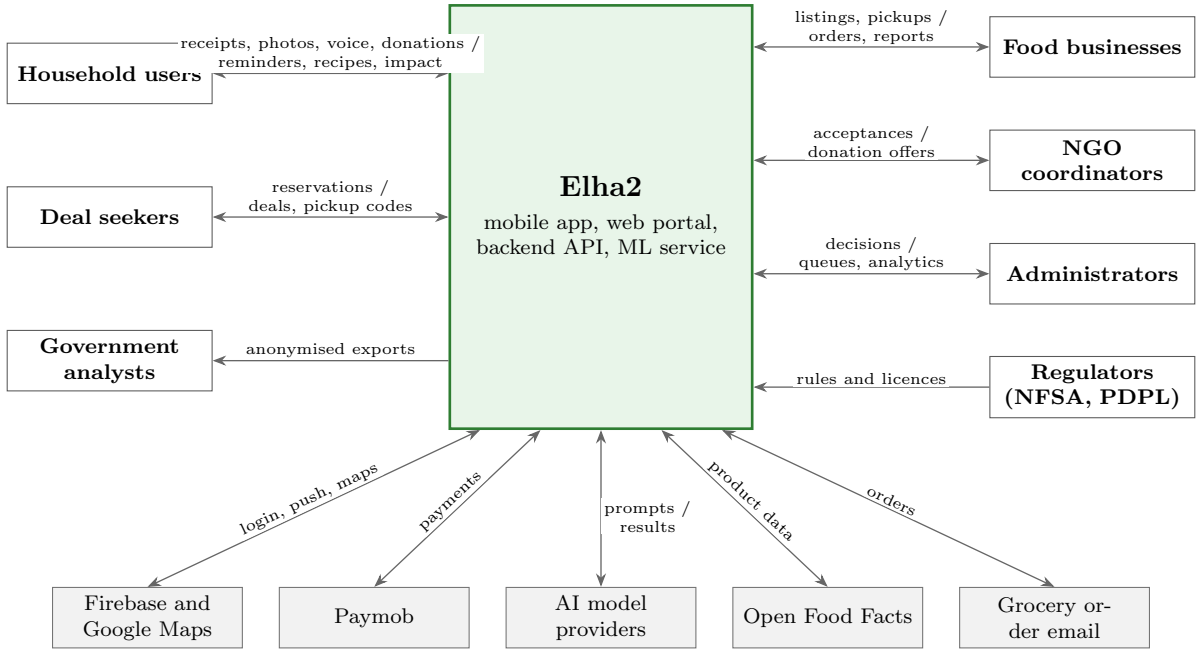


Figure 2.1: Context diagram: the Elha2 system boundary, the people and organisations it serves, and the external systems it depends on. Labels show the data sent in / out.

2.3 Design Summary

Elha2 is a client–server system with three tiers. Two clients, a Flutter mobile app and a React web portal, talk to one NestJS backend over HTTPS and WebSockets. The backend is a *modular monolith*: one deployable service divided into eight domain modules and five platform services that interact only through declared interfaces. A separate Python service runs the machine-learning models. PostgreSQL with PostGIS stores all data, locations and the job queue. Text and barcode recognition run on the phone, and every call to a hosted AI model goes through one AI gateway that validates results and falls back to a non-AI path on failure. Chapter 3 explains this design and the patterns it uses.

2.4 Design Stakeholders and Concerns

Each stakeholder concern below is answered by at least one part of the design.

Stakeholder	Design concern	Addressed in
Household users	Logging food with almost no typing; works offline; personal data stays private	AI subsystem (Section 3.2.2); on-device recognition (D3); offline store (Section 3.2.3)
Business staff	Fast listing; reliable pickup handover; never selling a bag twice	Marketplace and Orders modules (Section 3.2.1); atomic stock (D8)
NGO coordinators	Predictable, well-matched donation offers	Donations module and Scheduler (Section 3.2.1); routing algorithm (Section 5.9)
Administrators	Moderation and verification tools; rules that change without new code	Administration module; settings stored as data
Government partners	Trustworthy, anonymised impact data	Impact ledger (D7); Data design (Chapter 4)
Development team	Clear module boundaries for five people working in parallel; testable components	Modular monolith (D1); interface specifications (Section 3.2.4)
Instructor and TAs	Traceability from requirements to design; standard UML views	Requirements matrix (Chapter 7); UML component, class and sequence diagrams
Third-party providers	Respecting rate limits and terms of use	AI gateway cost monitor and cache (D4); integration adapters

3. System Architecture

3.1 Architectural Design

3.1.1 Subsystems and Responsibilities

The system is partitioned by *who uses it* and *how often it changes*. User-facing work is split between two clients, one for people on phones (households and deal seekers) and one for organisations on desktops (businesses, NGOs, administrators, analysts). All business rules live in one backend, so both clients behave the same and every rule is enforced in one place. Numeric machine learning, which needs Python libraries and long-running training jobs, is separated into its own service. External services sit behind interfaces that Elha2 owns. Figure 3.1 shows the major subsystems, the data repository and their interconnections as a UML component diagram.

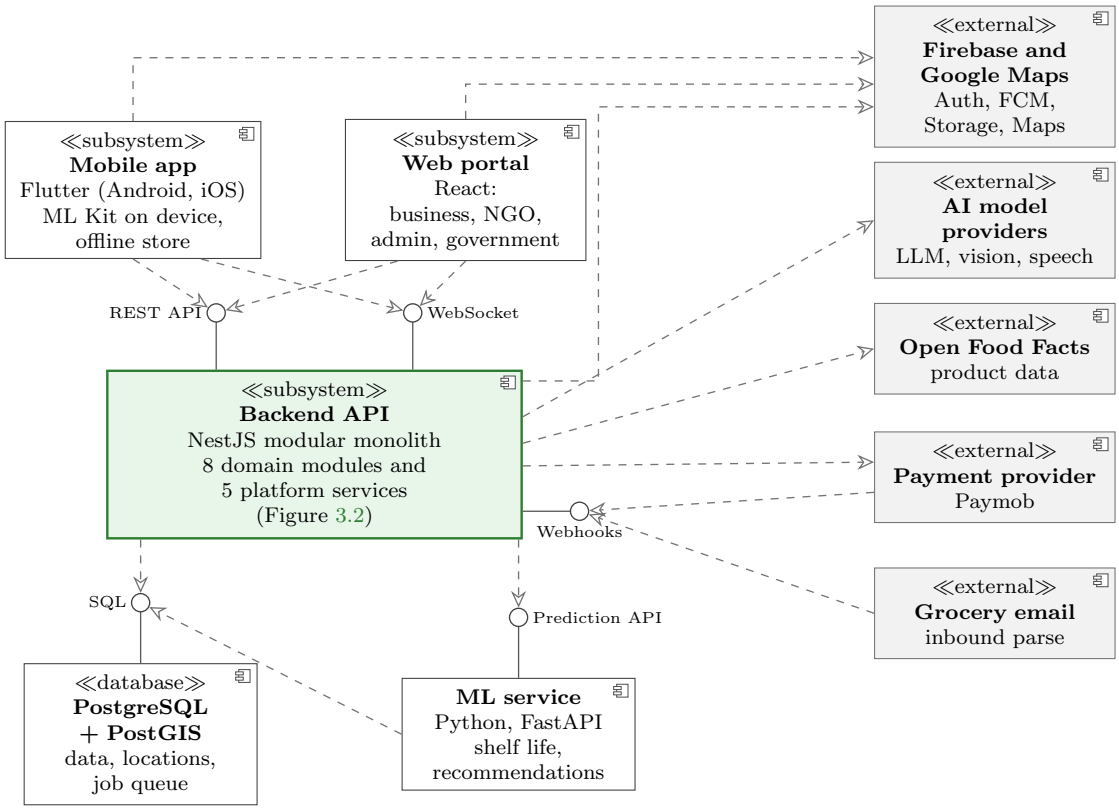


Figure 3.1: UML component diagram 1: system level. Balls are provided interfaces; dashed arrows are usage dependencies; grey components are external services.

Component	Responsibility	Technology	SRS trace
Mobile app	All household and deal seeker features, including household donations; on-device OCR, barcode scanning and personal-data masking; offline pantry	Flutter, ML Kit, SQLite (Drift), Riverpod	E1–E5, E7, E8–E11; NFR-16, NFR-21, NFR-27
Web portal	Business listings and pickups, NGO offers, administration, government exports	React, TypeScript	E6, E8, E9, E13
Backend API	Business logic, security and data access for both clients; webhooks from payment and email providers	NestJS (Node.js, TypeScript)	All epics; NFR-02, NFR-09, NFR-10
ML service	Shelf-life estimates and deal recommendations; nightly training	Python, FastAPI, scikit-learn	US-3.01, 5.06
PostgreSQL + PostGIS	Relational data, distance queries, job queue	PostgreSQL 16, PostGIS, pg-boss	SRS Section 6.1; NFR-03, NFR-20
Firebase and Google Maps	Phone login, push notifications, photo storage, maps and geocoding	Firebase Auth, FCM, Cloud Storage, Maps Platform	US-1.01, 5.01, 11.01
AI model providers	LLM, vision and speech models, reached only through the AI gateway	Hosted APIs (e.g. Claude, GPT or Gemini; Whisper)	Stories marked [AI]; NFR-07, NFR-22, NFR-29, NFR-30
Payment provider	Card and wallet payments; refunds; payouts	Paymob	E7; NFR-11
Open Food Facts, grocery email	Product data for barcodes; forwarded grocery orders	REST API, inbound email webhook	US-2.03, 2.04

3.1.2 How the Subsystems Collaborate

- **Clients to backend.** The mobile app and the web portal call the backend's REST API for every read and write, and keep a WebSocket open for live listing quantities and order status. Each request carries a Firebase ID token, which the backend verifies before routing.
- **Inside the backend.** Requests enter through the API layer and are handled by one domain module (Section 3.2.1). Domain modules call platform services (AI gateway, impact ledger, notifications, scheduler, adapters) and, where needed, each other, always through named interfaces.
- **Backend to ML service.** The backend calls the ML service over internal HTTP for shelf-life estimates and recommendations. The ML service reads its training data from a read-only database view and never writes business data.

- **Backend to external services.** Payments, maps, product data and inbound email go through the integration adapters; language, vision and speech models go through the AI gateway; push notifications go through Firebase Cloud Messaging.
- **Time-based work.** The scheduler starts jobs (daily digests, end of pickup window, offer time-outs, no-shows, nightly training) that call the owning module, so timed work follows the same rules as user requests.

3.1.3 Architectural Style and Design Patterns

Pattern	Where it is used	Why (quality attribute and SRS trace)
Architectural patterns		
Client–server, three tiers	Mobile app and web portal (presentation), backend and ML service (logic), PostgreSQL (data)	Rules are enforced once on the server; clients can change independently (NFR-10, NFR-27)
Layered architecture	Backend: API layer → domain modules → platform services → data. Mobile app: presentation → state → data → device services	Each layer depends only on the one below, which keeps code testable and replaceable (NFR-24)
Modular monolith	Eight domain modules in one NestJS service, each owning its data	Simple to run for a five-person team, with module boundaries that allow a later split (D1)
Offline-first with sync queue	Mobile pantry in SQLite; changes queued and replayed	Works on unstable mobile data (NFR-21, R-13)
Publish–subscribe	Domain modules publish events (order reserved, listing created, item expiring); Notifications and WebSocket gateway subscribe	Senders do not need to know who listens; live updates (US-5.03, E11)
Job queue (producer–consumer)	pg-boss jobs for digests, window ends, offer time-outs, training	Timed work is retried and committed with its data (NFR-19, D6)
Append-only ledger	Impact records, never updated or deleted	Every number is traceable and counted once (US-9.03, D7)
Design patterns [5]		
Facade (gateway)	AI gateway in front of all model providers; API layer in front of all modules	One place for prompts, validation, caching, cost limits and fallbacks (US-14.01, 14.05)
Adapter	Paymob, Google Maps, Open Food Facts, inbound email, ML service client, each behind an Elha2 interface	Vendors can be swapped without touching domain code (R-09, NFR-11)

Pattern	Where it is used	Why (quality attribute and SRS trace)
Strategy	AI provider and fallback strategy chosen per task in configuration; payment method (cash or Paymob) behind `IPayment`	New providers or payment methods are added as new strategies (NFR-22, E7)
Repository	Data access in each backend module and in the mobile data layer	Screens and business rules never touch SQL or the network directly (NFR-21, NFR-24)
Observer (state)	Riverpod providers notify Flutter widgets of state changes	Screens update automatically when data changes
Idempotent receiver	Idempotency keys on reservations and payments; impact records unique per source action	Retries never create a second order or a second impact record (NFR-06, US-9.03)
Template method	Every AI task runs the same steps (prompt, call, validate, log, fall back) with task-specific prompt and schema	Uniform safety and logging for all AI features (NFR-22, NFR-30)

3.1.4 Quality Attributes

The architecture was chosen to meet the non-functional requirements in SRS Chapter 5. The table lists the main tactics for each quality attribute.

Quality attribute	Architectural tactics	SRS requirements
Performance	On-device OCR and barcode scanning; PostGIS GiST index for radius search; caching of repeated AI results; static portal served from a CDN	NFR-01 to NFR-04
Scalability	Stateless API and ML containers behind a load balancer; scheduled work in a queue rather than in request threads	NFR-05
Integrity	Atomic conditional stock update; idempotency keys; one impact record per source action	NFR-06; US-9.03
Reliability	Job retries (3 attempts) with alerts; non-AI fallback for every AI task; daily encrypted backups; offline mobile store	NFR-19 to NFR-22
Availability	Managed database and container platform with health checks and automatic restarts	NFR-23
Security and privacy	Firebase token check and role guard on every request; TLS everywhere; personal data masked on the phone; each module owns its data; signed webhooks; no card data	NFR-09 to NFR-15
Safety	AI output filters; allergen checks and listing screening in the Marketplace module; only verified businesses can list	NFR-07, NFR-08

Quality attribute	Architectural tactics	SRS requirements
Usability	Arabic-first RTL layout in both clients; one-tap review of AI results; undo instead of confirmation for small changes	NFR-16 to NFR-18, NFR-30
Modifiability and testability	Module interfaces, adapters, strategies; configuration for rules and providers; OpenAPI generated from code; CI on every merge	NFR-24 to NFR-26
Cost	AI calls cached and capped per user; free tiers; one database for data and jobs	US-14.05

3.1.5 Technical Feasibility

Every part of the design uses mature, documented technology that the team can learn within the semester. All AI capabilities use existing models or services (SRS Section 2.2.1); we train only small tabular models. The stack runs on free tiers and student credits. The three riskiest technical questions (right-to-left layouts in Flutter, OCR accuracy on real Egyptian receipts, PostGIS distance queries) are answered by technical spikes in Sprint 1. Each of the five team members owns one area of the design (mobile app, web portal, backend domain modules, AI gateway and ML service, data and DevOps), and the declared interfaces let the five areas be built in parallel.

3.2 Decomposition Description

We use an **object-oriented description**. Section 3.2.1 decomposes the backend into modules, Section 3.2.2 the AI and ML subsystem and Section 3.2.3 the clients. Section 3.2.4 specifies the interfaces between them, and Section 3.2.5 shows how they collaborate in sequence diagrams. The object model (UML class diagrams, including the Organisation generalisation hierarchy and the aggregation of pantry items in a household) is in Chapter 4.

3.2.1 Backend API Subsystem

The backend has three layers (Figure 3.2). The *API layer* receives every request, checks the Firebase token and validates input. The eight *domain modules* hold the business rules, one per area of the backlog. The five *platform services* are shared by all domain modules. Modules call each other only through the named interfaces, which keeps the code testable and lets team members work on different modules in parallel.

Module	Responsibility	Epics	Owns data
Domain modules			
Accounts	Sign-up, profiles, households and members, organisations, branches, staff roles	E1	User, Household, Business, Branch, NGO

Module	Responsibility	Epics	Owns data
Pantry	Pantry items, receipt drafts, weekly check, status changes, waste reasons, daily expiry digests	E2, E3	PantryItem, ReceiptDraft
Expiry	Shelf-life estimates (through the ML service) and label types	E3	FoodCatalogueItem
Recipes	Recipe store, ranking by at-risk items, LLM fallback recipes, cooking assistant	E4	Recipe
Marketplace	Listings, stock, distance search, automatic listing screening	E5, E6, E12	Listing
Orders	Reservations, pickup codes, payments, refunds, payouts, no-show rule	E7	Order, Payment
Donations	NGO preferences, matching, offers, pickups, household donations, meals distributed	E8	DonationOffer
Administration	Verification, moderation queue, disputes, settings, platform analytics	E12, E13	Report, ModerationCase, Setting
Platform services			
AI gateway	Single entry point to all AI models (Section 3.2.2)	E14	PromptTemplate, AIRequestLog
Impact ledger	Append-only verified impact records, versioned factors, dashboards and exports	E9	ImpactRecord, ImpactFactor
Notifications	Push and in-app notifications, preferences, quiet hours	E11	Notification, DeviceToken
Scheduler	Timed jobs: daily digests, end-of-window routing, no-show handling, nightly model training	E3, E8	pg-boss job tables
Integration adapters	Paymob, Google Maps, Open Food Facts, inbound email, ML service client	E2, E6, E7	–

Module dependencies. The table lists the interfaces each module requires. Dependencies point only from domain modules to platform services, or between domain modules as shown in Figure 3.2, so the module graph has no cycles.

Module	Required interfaces
Accounts	INotify; IExternal (Firebase Admin token check)
Pantry	IEstimateExpiry, IAI, IRecordImpact, INotify; IExternal (IProduct, IInbound)
Expiry	IExternal (IPredict)
Recipes	IPantryQuery, IAI
Marketplace	IAI, INotify; IExternal (IGeo, IPredict)
Orders	IStock, IRecordImpact, INotify; IExternal (IPayment)
Donations	IStock, IRecordImpact, INotify; IExternal (IGeo)
Administration	IOrgVerify, INotify; IExternal (IPayment, for refunds)
Scheduler	Pantry (digests), Marketplace (window end), Donations (routing), Orders (no-shows)

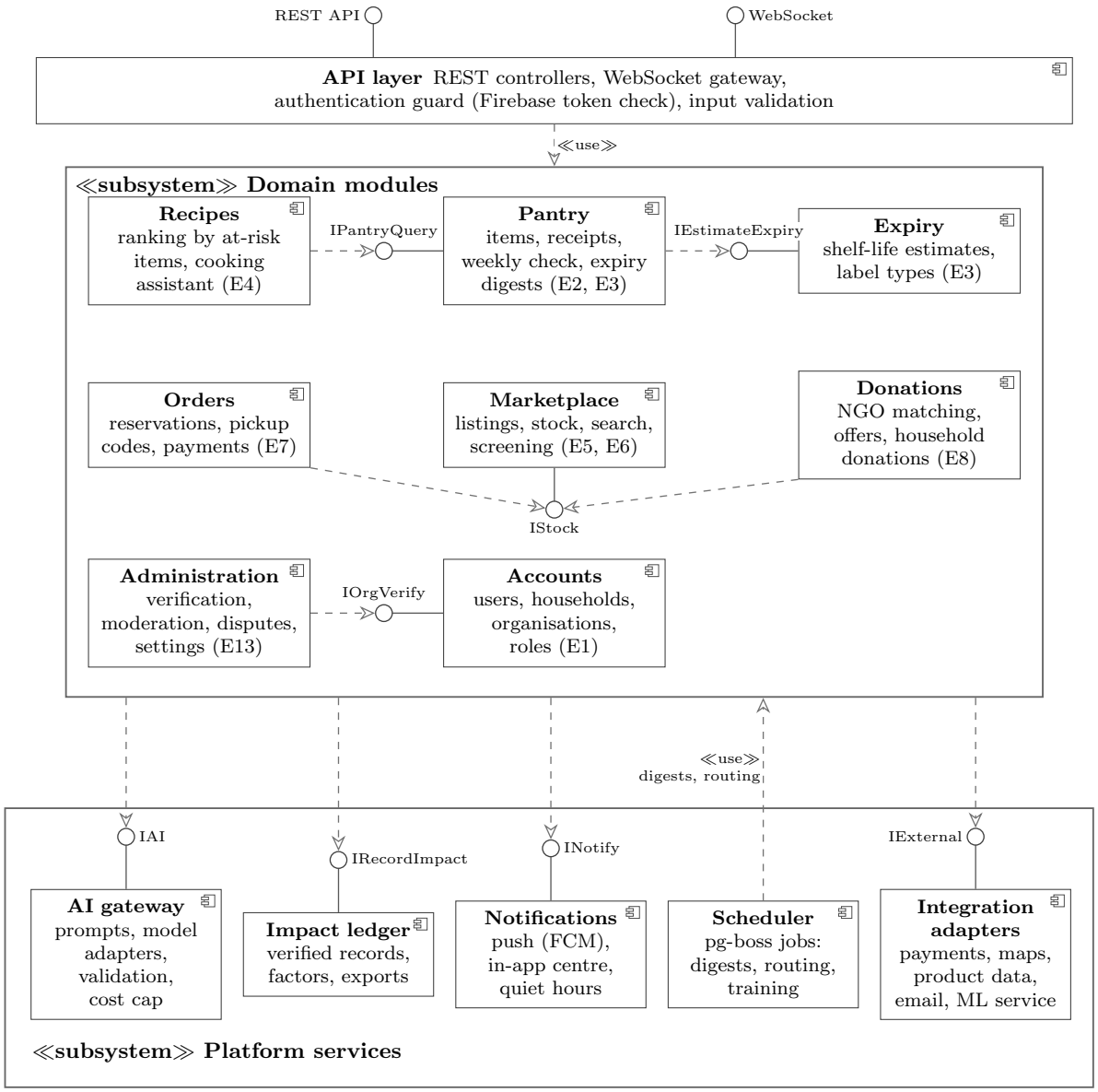


Figure 3.2: UML component diagram 2: inside the backend API.

3.2.2 AI and ML Subsystem

AI appears in two places (Figure 3.3). Language, vision and speech tasks go through the *AI gateway* to hosted model APIs. Numeric predictions (shelf life, surplus forecasts and recommendations) come from our own *ML service*. Text and barcode recognition run on the phone, so they work offline, and personal data is masked before anything leaves the device.

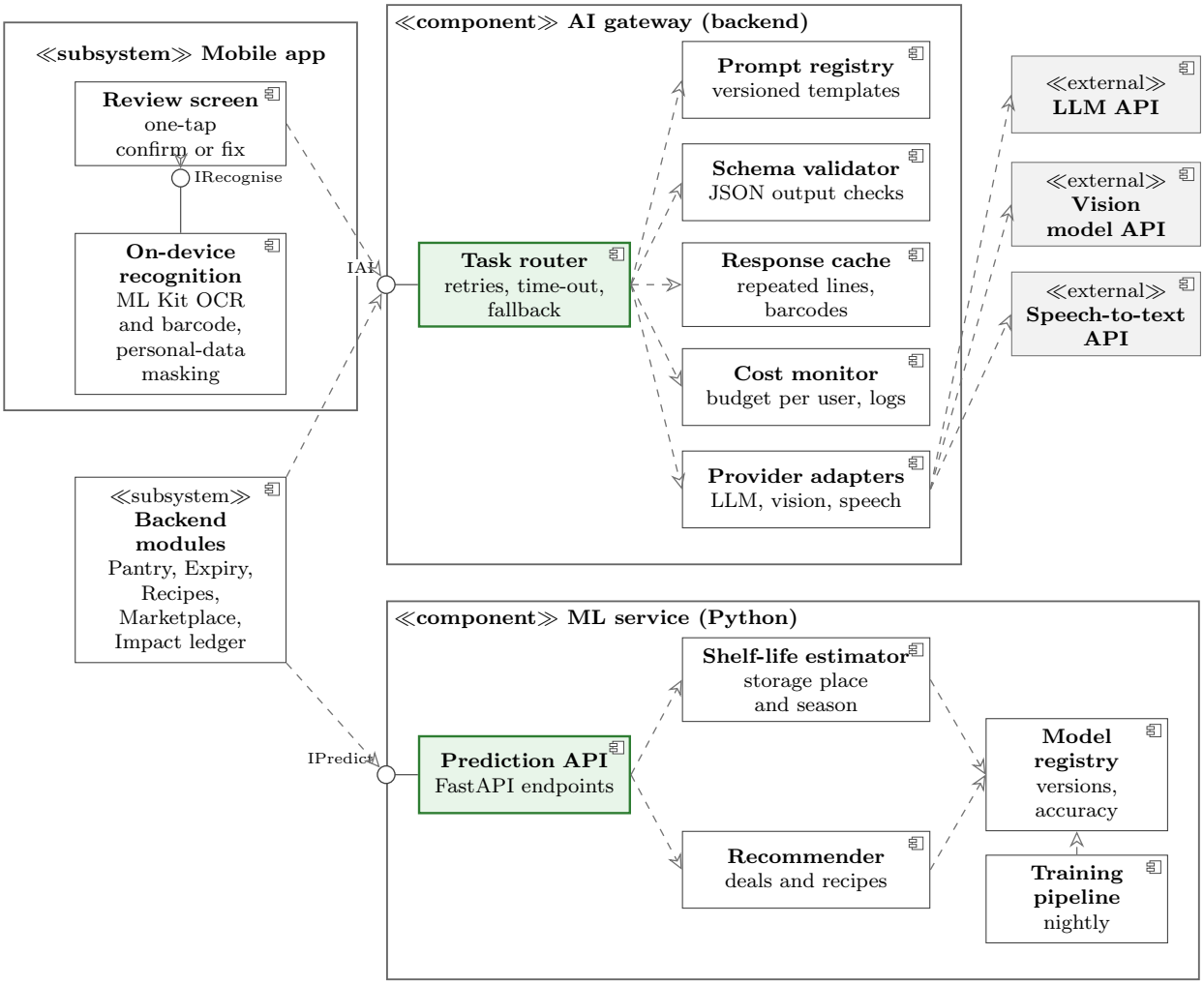


Figure 3.3: UML component diagram 3: AI and ML subsystem across the phone, the backend and the ML service.

AI task	Model	Output and safeguard	Stories
Read receipt text	ML Kit OCR (on device)	Text lines; personal data masked before upload	US-2.01, 14.03
Parse receipt or voice note	Hosted LLM	JSON items with confidence; schema check; user review; manual fallback	US-2.01, 2.02, 2.06
Recognise items in a photo	Vision-language model	Item list with counts; user review; photo deleted within 24 h	US-2.05, 6.02

AI task	Model	Output and safeguard	Stories
Transcribe Egyptian Arabic	Speech-to-text (e.g. Whisper)	Transcript passed to the LLM parser	US-2.06
Estimate shelf life	ML service: reference rules adjusted by a regression model	Date marked “estimated”; user can override	US-3.01
Suggest recipes	Curated recipe ranking; LLM fallback	Food-safety filter; no medical advice	US-4.01, 4.02
Summarise waste	Hosted LLM over the user’s own statistics	At most 3 tips, each backed by data	US-9.05

The task router applies the same rules to every AI call: a 15 s time-out, one retry, schema validation of the response, and a non-AI fallback when the call fails (NFR-22). It also logs model version, latency, cost and confidence for the accuracy report (US-14.02) and the cost cap (US-14.05). The model provider is chosen in configuration, so it can be changed without touching the domain modules.

3.2.3 Mobile App and Web Portal

The mobile app uses a layered structure so that screens never call the network directly. This keeps the offline pantry and the sync logic in one place.

Layer	Components
Presentation	Screens for the five tabs (Home, Pantry, Deals, Donate, Impact) and Settings; Arabic and English with right-to-left layout
State	Riverpod providers holding screen state and user actions
Data	Repositories; REST and WebSocket API client; offline store (SQLite through Drift); sync queue that replays offline changes
Device services	ML Kit recognition and personal-data masking; camera and microphone; push handler (FCM); Google Maps view; secure token storage

The web portal follows the same split: React pages, a shared API client, and role-based routes for businesses, NGOs, administrators and government analysts.

3.2.4 Interface Specifications

External API (clients to backend). Both clients use one versioned REST API (`/api/v1`) with JSON bodies. Every request carries a Firebase ID token, which the API layer verifies before routing; roles decide what each user may do (NFR-10). Errors use one format (code, message, field

errors) in the user's language. Reservation and payment requests carry an idempotency key, so a retried request never creates a second order. Live updates (listing quantities, chat, order status) are pushed over a WebSocket connection.

Resource	Main endpoints	Clients	Stories
`/me`, `/households`	Profile, language, location; household members and invitations	Mobile	E1
`/organisations`	Register business or NGO, branches, staff, documents	Portal	US-1.06, 1.07, 6.03
`/pantry/items`	List, add, update status, delete; search	Mobile	US-2.07–2.11
`/pantry/receipts`	Upload masked lines, get draft items, confirm	Mobile	US-2.01, 2.02
`/pantry/check`	Get this week's check cards, submit swipes	Mobile	US-2.09
`/recipes`	Suggestions for at-risk items, filters, mark as cooked	Mobile	E4
`/listings`	Nearby search (radius, filters), create, edit, cancel	Both	E5, E6
`/orders`	Reserve, cancel, history, confirm pickup by code	Both	E7
`/donations`	Offers for an NGO, accept or decline, book pickup, record meals; household donations to drop-off points	Both	E8
`/impact`	Personal, business and NGO dashboards; calculation method	Both	E9
`/exports`	CSR report (PDF), anonymised government data (CSV)	Portal	US-9.08, 9.10
`/admin`	Verification, moderation queue, disputes, settings, analytics	Portal	E12, E13
`/webhooks`	Payment status (Paymob), inbound grocery email	Providers	E7, US-2.04

Internal interfaces.

Interface	Provider	Main operations	Used by
IPantryQuery	Pantry	`atRiskItems(household, days)`, `deduct(items)`	Recipes
IEstimateExpiry	Expiry	`estimate(item, storage, purchased)` returns date, source and confidence	Pantry
IStock	Marketplace	`reserve(listing, qty)` (atomic), `release(order)`, `unsoldAtWindowEnd(listing)`	Orders, Donations
IOrgVerify	Accounts	`setVerification(org, status, reason)`	Administration
IAI	AI gateway	`parseReceipt`, `parseVoice`, `recognisePhoto`, `suggestRecipe`, `summariseWaste`; each returns a result, a confidence and a fallback flag	Pantry, Recipes, Marketplace, Impact ledger
IRecordImpact	Impact ledger	`record(sourceAction, items, parties)`, idempotent per source action	Pantry, Orders, Donations
INotify	Notifications	`send(user, type, payload)`, which respects preferences and quiet hours	All modules

Interface	Provider	Main operations	Used by
IExternal	Integration adapters	IPayment (charge, refund, verify webhook), IGeo (geocode, distance), IProduct (barcode lookup), IPredict (shelf life, recommend), IInbound (parse email)	See Section 3.2.1
IPredict (HTTP)	ML service	`POST /shelf-life`, `POST /recommend`	Integration adapters

External service interfaces (Firebase, Paymob, Google Maps, model providers, Open Food Facts, inbound email) are listed in SRS Section 3.3. Each one is wrapped by an adapter, so the rest of the code never depends on a vendor SDK directly.

3.2.5 Sequence Diagrams

Scan a receipt (UC-05). Figure 3.4 shows how the components of Figures 3.1 to 3.3 work together when a household user scans a receipt. OCR and masking run on the phone, so only masked text reaches the backend. The AI gateway calls the LLM, validates the reply and records its cost before the Pantry module adds expiry estimates. Nothing is saved until the user confirms.

Reserve and collect a bag (UC-22, UC-25). Figure 3.5 shows the marketplace handover. The stock update is atomic, so two buyers can never get the last bag. At pickup, the Orders module checks the code against the branch and asks the Impact ledger to record the saving exactly once.

Route unsold food to an NGO (UC-33). This flow is started by the scheduler rather than by a user. Its algorithm is given in Section 5.9.

1. When a listing is created, the Scheduler queues a job for the end of its pickup window.
2. At that time, the job calls `IStock.unsoldAtWindowEnd`; if anything is left and auto-donation is on, it calls Donations.
3. Donations finds verified NGOs within range whose food types, capacity and hours match, ordered with the business's default partner first.
4. It sends an offer to the first NGO and queues a 30-minute expiry job; if the offer is declined or expires, it moves to the next NGO.
5. When an NGO accepts, the business is notified; the handover is confirmed by code and recorded in the Impact ledger.

3.3 Design Rationale

3.3.1 Architectures Considered

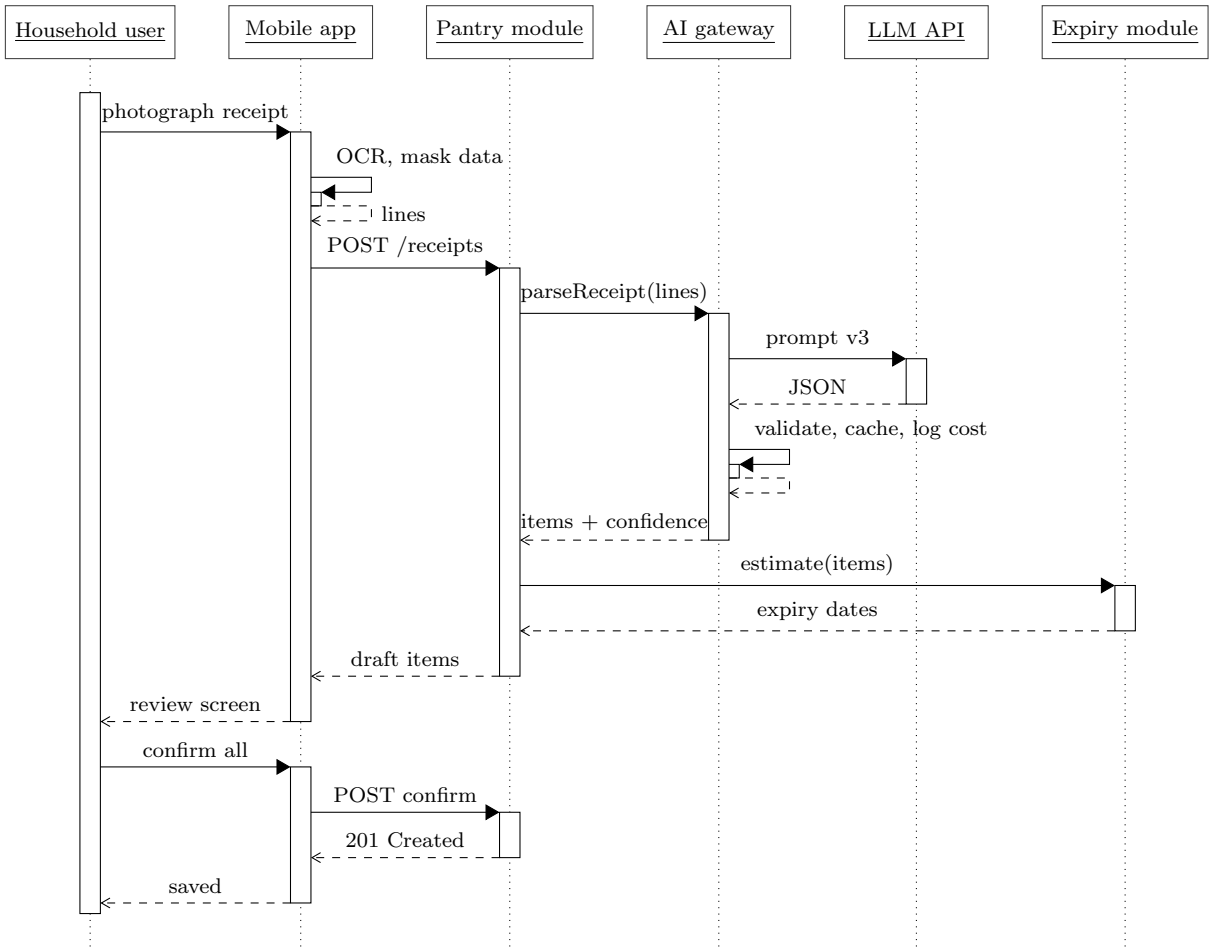


Figure 3.4: UML sequence diagram: scanning a receipt (SRS UC-05).

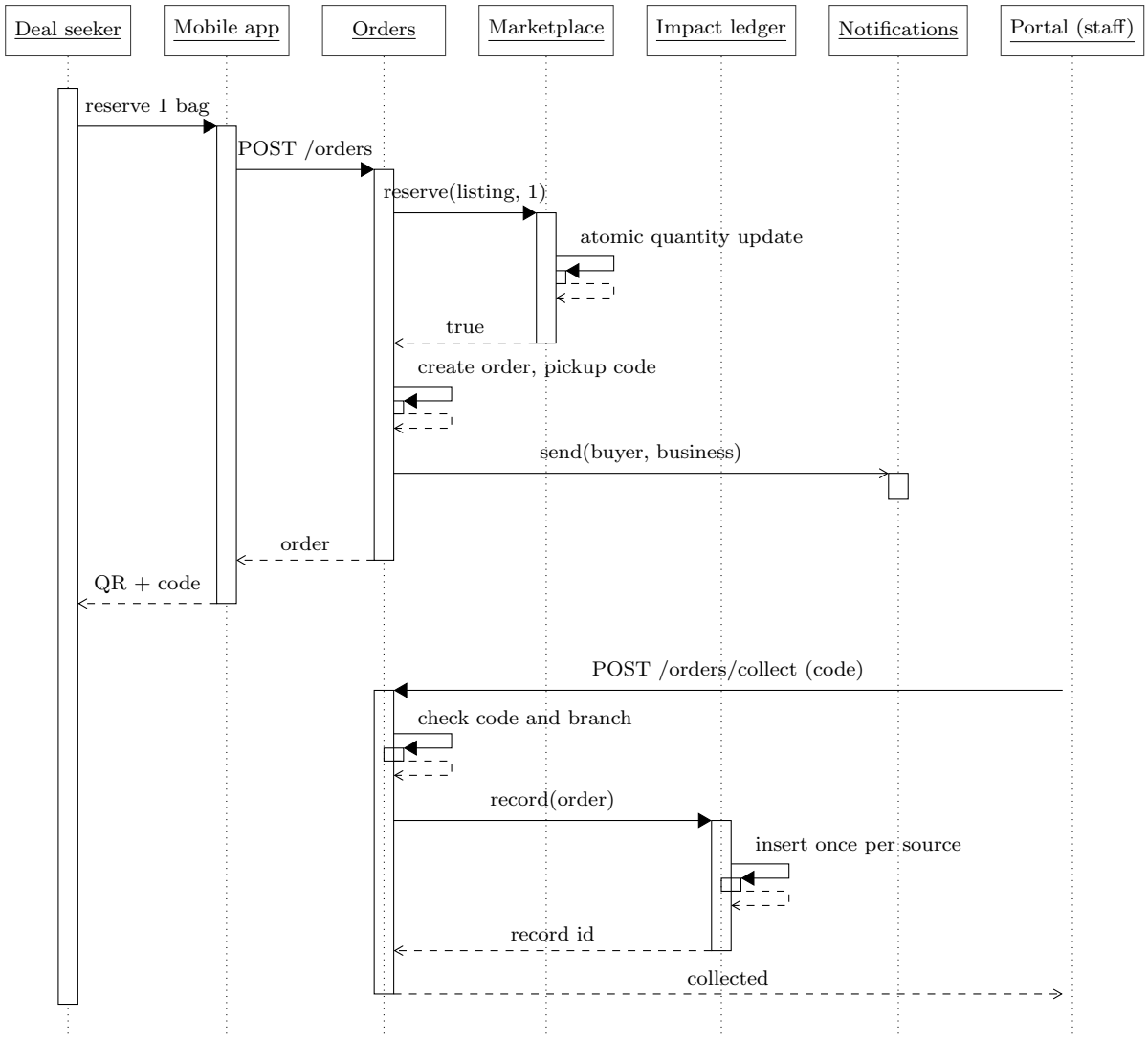


Figure 3.5: UML sequence diagram: reserving a bag and collecting it with the pickup code (SRS UC-22 and UC-25).

Architecture	Advantages	Why it was not chosen
Microservices (one service per feature)	Independent deployment and scaling; strong isolation	Too much operational work for five students in one semester (networking, tracing, many deployments, distributed transactions for stock and impact). The modular monolith keeps the same boundaries without the cost.
Backend-as-a-service only (Firebase Firestore and Cloud Functions, no custom backend)	Very fast to start; no server to run	Firestore cannot do PostGIS radius queries or the transactional stock and ledger rules we need, and business rules would be spread across many small functions.
Two separate backends (one for the app, one for the portal)	Each client gets an API tailored to it	Duplicates every business rule and the data access; the same rules must apply to both clients.
Chosen: modular monolith + Python ML service	One deployable, transactional backend with clear module boundaries; ML in the language with the best libraries	Trade-off: the whole backend is deployed together, and one faulty module can affect others. Mitigated by tests in CI and module interfaces that allow a module to be split out later.

3.3.2 Design Decisions

ID	Decision	Alternatives considered	Reason and trade-off (SRS trace)
D1	Modular monolith backend	Microservices	A team of five in one semester cannot run many services. Declared module interfaces still allow a module to be split out later. (NFR-24)
D2	Flutter for Android and iOS	Native apps; React Native	One codebase, strong right-to-left support, chosen in Phase 1. (NFR-16, NFR-27)
D3	OCR and personal-data masking on the phone	Send the photo to a cloud OCR service	Works offline, sends less data, and keeps personal details off third-party servers. (NFR-14, NFR-21, US-14.03, R-02, R-07)
D4	One AI gateway for every model call	Each module calls providers directly	One place for prompts, validation, caching, cost limits and fallbacks; the provider can be changed in configuration. (US-14.01, NFR-22, R-09)
D5	Separate Python ML service	Run ML inside Node.js	ML libraries are mature in Python; heavy jobs stay away from request handling. (US-3.01, 5.06)
D6	Job queue in PostgreSQL (pg-boss)	Redis with BullMQ	One less service to run; jobs commit in the same transaction as their data. (NFR-19)
D7	Append-only, versioned impact ledger	Compute totals on the fly from orders	Every number can be traced to a verified action and a factor version, and nothing is counted twice. (US-9.02, 9.03, R-11)

ID	Decision	Alternatives considered	Reason and trade-off (SRS trace)
D8	Atomic conditional stock update	Lock in application code	The database guarantees no overselling, even with many simultaneous requests. (NFR-06)
D9	A person confirms every AI output	Fully automatic AI	AI can be wrong; confirmation keeps users in control and builds trust. (NFR-30, R-01, R-12)
D10	Firebase phone-number login	Email and password	Egyptian users rely on mobile numbers; no passwords to forget; free tier. (US-1.01)

3.3.3 Open Design Question

- Which hosted LLM to use, chosen in Sprint 1 by comparing accuracy and cost on our Egyptian receipt test set (SRS TBD-01).

4. Data Design

4.1 Data Description

4.1.1 Data Stores

Store	Contents	Technology
Main database	All business entities (Figures 4.1 and 4.2), locations, settings, impact ledger	PostgreSQL 16 with PostGIS; one schema per backend module
Job queue	Scheduled and retried jobs	pg-boss tables in the same PostgreSQL database
Offline store (phone)	The household's pantry, catalogue cache, queued changes	SQLite through Drift
File storage	Organisation documents; listing photos; AI photos (deleted within 24 h)	Firebase Cloud Storage
ML artefacts	Trained shelf-life and recommendation models, feature snapshots	Files in the ML service's storage bucket, versioned by date
AI test set	Labelled receipts, photos and voice notes (US-14.02)	Private repository folder, never shipped in the app
Logs and metrics	Request logs, errors, AI call logs (no personal data)	Platform logging; `AIRequestLog` table

4.1.2 From Information Domain to Data Structures

Each real-world thing in the SRS (a person, a household, a pantry item, a business, a listing, an order, a donation, a saving) becomes one class in the object model below and one table in PostgreSQL, mapped by the ORM. Each class is owned by exactly one backend module (Section 3.2.1); other modules read or change it only through the owner's interface, never through its tables. Enumerations are stored as PostgreSQL enum types, money as integer piastres with the currency, and locations as PostGIS `geography(Point)`. The logical entities and their integrity rules are listed in SRS Section 6.1.

Figure 4.1 shows the household side of the model and the impact ledger; Figure 4.2 shows organisations, the marketplace and donation, including the *Organisation* generalisation hierarchy (Business and NGO) and the composition of branches and listings.

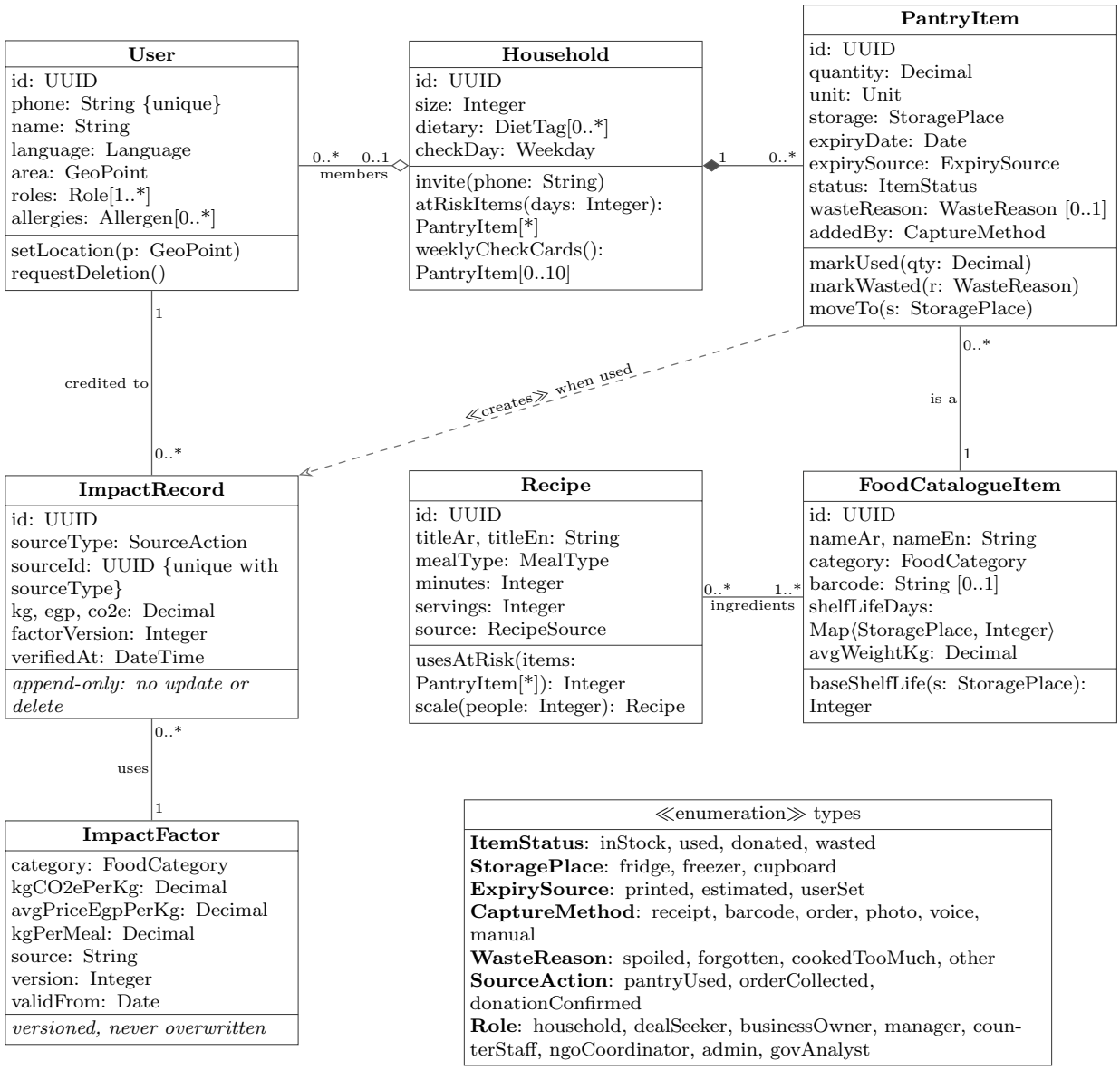


Figure 4.1: UML class diagram 1: users, households, the pantry, recipes and the impact ledger. Filled diamond = composition; hollow diamond = aggregation.

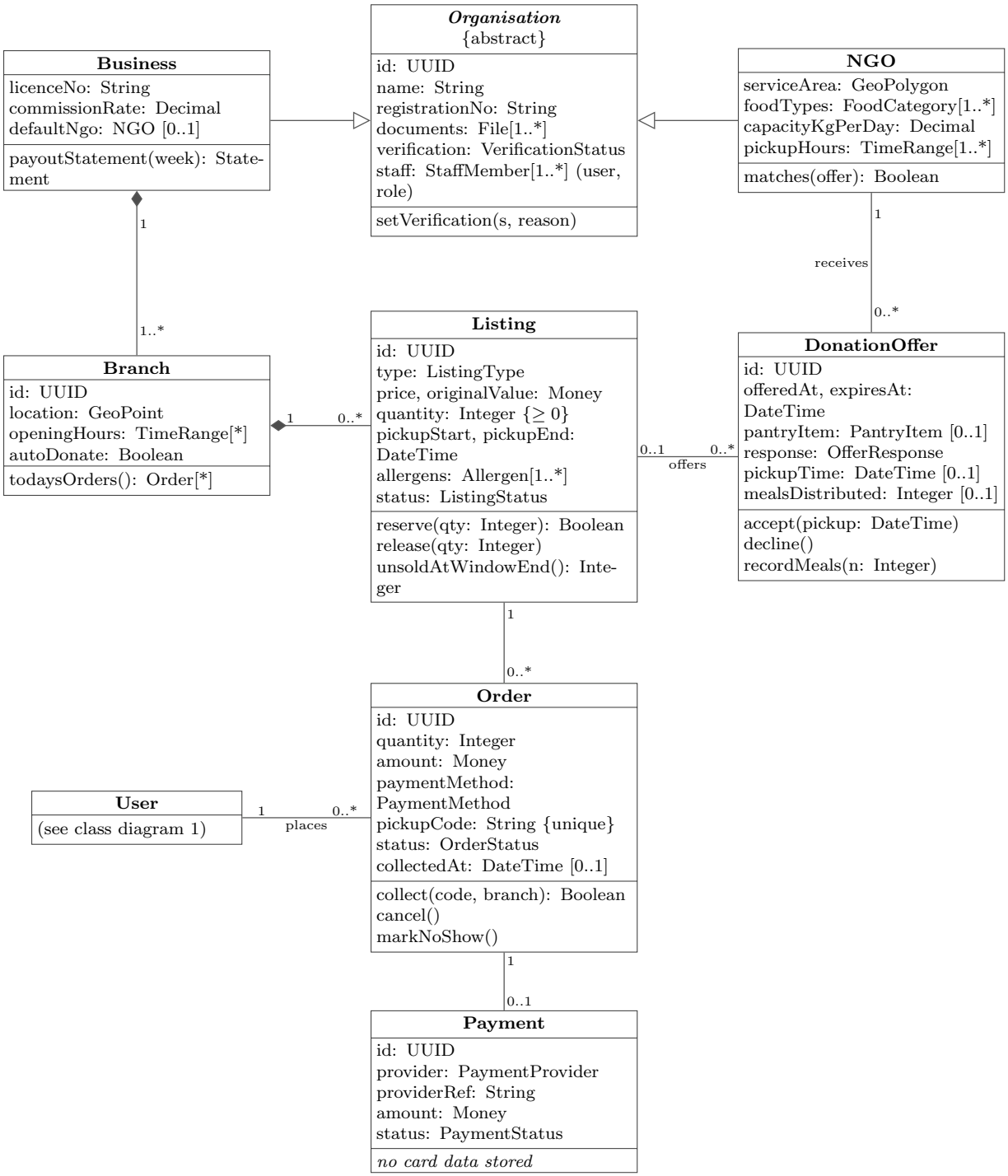


Figure 4.2: UML class diagram 2: organisations, marketplace and donation. Hollow triangles = generalisation; *Organisation* is abstract.

4.1.3 Key Data Design Decisions

- **Locations.** Listings, branches, NGOs and drop-off points store a PostGIS `geography(Point)` column with a GiST index, so radius searches stay under 2 s (NFR-03). Public responses about users return only a distance and an area name, never an exact point.
- **Impact ledger.** `ImpactRecord` rows are append-only and unique per source action (an order, a donation or a pantry item). Each row stores the factor version it used, so totals never change when factors are updated, and nothing is counted twice (US-9.03). `ImpactFactor` rows are versioned and never overwritten (US-9.02).
- **Stock.** A reservation reduces `Listing.quantity` in one conditional update (`quantity >= requested`) inside a transaction, which guarantees that a bag is never sold twice (NFR-06).
- **Offline copy.** The mobile app keeps the household's pantry in SQLite. Changes are queued with timestamps and merged per field on reconnect, and every change can be undone (NFR-21, R-13; algorithm in Section 5.11).
- **Personal data.** Card data is never stored (NFR-11). AI photos live in Cloud Storage under a 24-hour deletion rule (NFR-14). `AIRequestLog` stores no personal data. Account deletion removes personal fields within 30 days and keeps only anonymised impact rows (US-1.08).
- **Jobs.** pg-boss keeps scheduled jobs in PostgreSQL, so a job and the data it changes can be committed in the same transaction, and failed jobs are retried (NFR-19).
- **Backups.** Daily encrypted backups, with a 24-hour recovery point and a 4-hour recovery time (NFR-20).

4.2 Data Dictionary

The main classes in alphabetical order, with their attributes and types, their operations and the module that owns them. Every class also has `createdAt` and `updatedAt` timestamps, not repeated below. Enumeration types are listed in Figure 4.1.

Class	Attributes (name: type)	Operations	Description and owner
AIRequestLog	id: UUID; feature: AITask; modelVersion: String; promptVersion: Integer; latencyMs: Integer; costUsd: Decimal; confidence: Decimal; fallbackUsed: Boolean; userCorrected: Boolean; createdAt: DateTime	–	One row per AI call; holds no personal data (US-14.04, 14.05). *Owner: AI gateway.*
Branch	id: UUID; business: Business; name: String; location: GeoPoint; address: String; openingHours: TimeRange[*]; autoDonate: Boolean	todaysOrders(): Order[*]	A business location that lists surplus (US-1.06, 6.07). *Owner: Accounts.*

Class	Attributes (name: type)	Operations	Description and owner
Business	(inherits Organisation) licenceNo: String; commissionRate: Decimal; defaultNgo: NGO [0..1]	payoutStatement(week: Week): Statement	Food business; may list only when verified (BR-01). *Owner: Accounts.*
DonationOffer	id: UUID; ngo: NGO; listing: Listing [0..1]; pantryItem: PantryItem [0..1]; offeredAt, expiresAt: DateTime; response: OfferResponse; pickupTime: DateTime [0..1]; mealsDistributed: Integer [0..1]	accept(t: DateTime); decline(); recordMeals(n: Integer)	An offer of unsold or donated food to one NGO (US-8.02, 8.04, 8.07). *Owner: Donations.*
FoodCatalogue-Item	id: UUID; nameAr, nameEn: String; category: FoodCategory; barcode: String [0..1]; shelfLifeDays: Map(StoragePlace → Integer); avgWeightKg: Decimal	baseShelfLife(s: StoragePlace): Integer	Reference food data managed by administrators (US-13.06). *Owner: Expiry.*
Household	id: UUID; owner: User; members: User[1..*]; size: Integer; dietary: DietTag[*]; checkDay: Weekday	invite(phone: String); atRiskItems(days: Integer): PantryItem[*]; weeklyCheckCards(): PantryItem[0..10]	A shared pantry and its members (US-1.05). *Owner: Accounts.*
ImpactFactor	category: FoodCategory; kgCO2ePerKg: Decimal; avgPriceEgpPerKg: Decimal; kgPerMeal: Decimal; source: String; version: Integer; validFrom: Date	current(c: FoodCategory): ImpactFactor	Versioned conversion factors; never overwritten (US-9.02). *Owner: Impact ledger.*
ImpactRecord	id: UUID; sourceType: SourceAction; sourceId: UUID; user: User [0..1]; business: Business [0..1]; ngo: NGO [0..1]; category: FoodCategory; kg, egp, co2e: Decimal; meals: Decimal; factorVersion: Integer; verifiedAt: DateTime	–	One verified saving; unique on (sourceType, sourceId); append-only (US-9.03). *Owner: Impact ledger.*
Listing	id: UUID; branch: Branch; type: ListingType; title: String; price, originalValue: Money; quantity: Integer (≥ 0); pickupStart, pickupEnd: DateTime; allergens: Allergen[1..*]; photoUrl: URL; status: ListingStatus	reserve(qty: Integer): Boolean; release(qty: Integer); unsol-dAtWindowEnd(): Integer	Surplus offered for pickup (US-6.01, 5.04). *Owner: Marketplace.*
NGO	(inherits Organisation) serviceArea: GeoPolygon; foodTypes: FoodCategory[1..*]; capacityKgPerDay: Decimal; pickupHours: TimeRange[1..*]	matches(offer: DonationOffer): Boolean	Verified charity or food bank (US-1.07, 8.01). *Owner: Accounts, Donations.*

Class	Attributes (name: type)	Operations	Description and owner
Notification	id: UUID; user: User; type: NotificationType; title, body: String; deepLink: String; sentAt: DateTime; readAt: DateTime [0..1]	markRead()	In-app and push notification; kept 30 days (US-11.05). *Owner: Notifications.*
Order	id: UUID; buyer: User; listing: Listing; quantity: Integer; amount: Money; paymentMethod: PaymentMethod; pickupCode: String (unique, 4 digits + QR token); idempotencyKey: String (unique); status: OrderStatus; collectedAt: DateTime [0..1]	collect(code: String, branch: Branch): Boolean; cancel(); markNoShow()	A reservation of surplus (US-5.04, 7.03). *Owner: Orders.*
Organisation	*abstract*; id: UUID; name: String; registrationNo: String; documents: File[1..*]; verification: VerificationStatus; staff: StaffMember[1..*]	setVerification(s: VerificationStatus, reason: String)	Generalisation of Business and NGO (US-13.01). *Owner: Accounts.*
PantryItem	id: UUID; household: Household; catalogueItem: FoodCatalogueItem; quantity: Decimal; unit: Unit; storage: StoragePlace; expiryDate: Date; expirySource: ExpirySource; status: ItemStatus; wasteReason: WasteReason [0..1]; addedBy: CaptureMethod; updatedAt: DateTime	markUsed(q: Decimal); markWasted(r: WasteReason); moveTo(s: StoragePlace)	One food item in a household pantry (E2). *Owner: Pantry.*
Payment	id: UUID; order: Order; provider: PaymentProvider; providerRef: String; amount: Money; status: PaymentStatus	refund(amount: Money)	Online payment record; no card data (NFR-11). *Owner: Orders.*
ReceiptDraft	id: UUID; household: Household; lines: String[*] (masked); items: DraftItem[*] (name, qty, unit, category, confidence); createdAt: DateTime	confirm(edits: DraftItem[*]): PantryItem[*]	AI-extracted items awaiting user review; deleted after 24 h (US-2.02). *Owner: Pantry.*
Recipe	id: UUID; titleAr, titleEn: String; ingredients: (FoodCatalogueItem, quantity)[1..*]; steps: String[*]; mealType: MealType; minutes: Integer; servings: Integer; source: RecipeSource (curated or LLM)	usesAtRisk(items: PantryItem[*]): Integer; scale(people: Integer): Recipe	Curated Egyptian recipe or checked LLM recipe (E4). *Owner: Recipes.*
Setting	key: String; value: JSON; version: Integer; changedBy: User; changedAt: DateTime	–	Configurable rule such as discount limits or time limits (US-13.04). *Owner: Administration.*

Class	Attributes (name: type)	Operations	Description and owner
User	id: UUID; firebaseUid: String (unique); phone: String (unique); name: String; language: Language; area: GeoPoint; roles: Role[1..*]; allergies: Allergen[*]; notificationPrefs: JSON; deletedAt: DateTime [0..1]	setLocation(p: GeoPoint); requestDeletion()	Any registered person (E1). *Owner: Accounts.*

5. Component Design

This chapter takes a closer look at what each component does. For the operations behind the stories we implement during the course, it gives the algorithm in pseudocode (PDL) with the local data it uses. Operations that only read or write one record (for example editing a profile) are not repeated here. The pseudocode is refined into code during each sprint; when the implementation differs, this chapter is updated with change markings.

Conventions: CamelCase names are operations of the component; `tx` is a database transaction; `config` is a platform setting changeable by administrators (US-13.04); comments start with ▷.

5.1 AI Gateway: Run an AI Task

Used by every AI feature (US-14.01, NFR-22, NFR-30). Local data: the versioned prompt template and JSON schema for the task, and a cache keyed by a hash of the input.

Algorithm 1 RunTask(task, input, user)

Input: task name, input (already masked on the phone), calling user
Output: result, confidence, fallbackUsed

```
template ← PromptTemplate(task, config.promptVersion[task])
key ← Hash(task, template.version, input)
if cache contains key then return cache[key]
if MonthlyCost(user) ≥ config.aiBudgetPerUser then return (Fallback(task, input), 0, true)
for attempt ← 1 to 2 do                                              ▷ one retry
    reply ← Provider(config.provider[task]).call(template.fill(input), timeout = 15 s)
    if reply arrived and reply matches template.schema and SafetyFilter(task, reply) passes then
        Log(task, model, latency, cost, reply.confidence, fallback = false)
        cache[key] ← reply; return (reply.result, reply.confidence, false)
Log(task, model, latency, cost, 0, fallback = true)
return (Fallback(task, input), 0, true)                  ▷ e.g. raw OCR lines for manual entry
```

5.2 Pantry: Scan and Confirm a Receipt

US-2.01, 2.02, 14.03. The first part runs on the phone; the rest in the Pantry module. Local data: the receipt draft (masked lines and draft items).

Algorithm 2 ScanReceipt(photo) and ConfirmDraft(draftId, edits)

```
lines ← MLKitOCR(photo)                                          ▷ on the phone
lines ← MaskPersonalData(lines)                  ▷ names, phone numbers, card digits
if phone is offline then queue (lines) for upload; return
(items, conf, fallback) ← AIGateway.RunTask("parseReceipt", lines, user)
if fallback then return draft with lines only                    ▷ user adds items by hand
for all item in items do
    item.catalogue ← MatchCatalogue(item.name)
    (item.expiry, item.source) ← Expiry.Estimate(item.catalogue, item.storage, today)
save ReceiptDraft(lines, items); return items with confidences                ▷ review screen

procedure ConfirmDraft(draftId, edits)
    begin tx
    for all item in draft.items updated by edits, not deleted do
        insert PantryItem(household, item, status = inStock, addedBy = receipt)
    delete draft; commit tx
    LogCorrections(draft, edits)                          ▷ no personal data (US-14.04)
```

5.3 Expiry: Estimate an Expiry Date

US-3.01. Local data: the catalogue's base shelf life per storage place; a seasonal factor table. The ML service refines the rule-based estimate when it has enough data.

Algorithm 3 Estimate(catalogueItem, storage, purchasedOn)

Output: expiry date and its source

```
if a printed date was captured then return (printedDate, printed)
days ← catalogueItem.baseShelfLife(storage)
if storage ≠ freezer and month of purchasedOn ∈ {May, ..., September} then
    days ← ⌊ days × config.summerFactor ⌋                  ▷ shorter in Egyptian summer
(mlDays, ok) ← MLService.ShelfLife(catalogueItem, storage, purchasedOn)      ▷ optional
if ok then days ← Min(days, mlDays)                                ▷ be conservative
return (purchasedOn + days, estimated)
```

5.4 Pantry: Choose the Weekly Check Cards

US-2.09. Local data: for each item, the expected usage date from its category's typical consumption time.

Algorithm 4 WeeklyCheckCards(household)

candidates ← items of household with status = inStock
for all item **in** candidates **do**
    age ← today − item.addedOn
    uncertainty(item) ← age / expectedUseDays(item.category) ▷ higher = more likely already used
    if today > item.expiryDate + config.graceDays **then** mark item archived; remove from candidates
return the 10 candidates with the highest uncertainty ▷ at most 10 cards

5.5 Recipes: Suggest Recipes for At-Risk Food

US-4.01, 4.05. Local data: the curated recipe table.

Algorithm 5 Suggest(household) and MarkCooked(recipe)

atRisk ← Pantry.AtRiskItems(household, days = 3)
ranked ← curated recipes sorted by usesAtRisk(atRisk) (descending), then minutes
top ← first 5 of ranked with usesAtRisk ≥ 1, filtered by household.dietary
if |top| < 5 **then**
    (extra, conf, fallback) ← AIGateway.RunTask("suggestRecipe", atRisk, user)
    if not fallback **then** append extra (marked "AI suggestion") to top
return top

procedure MarkCooked(recipe)
    Pantry.Deduct(recipe.ingredients scaled to household.size) ▷ undo available for 10 s
    ImpactLedger.Record(pantryUsed, used items, household user)

5.6 Marketplace and Orders: Reserve a Bag

US-5.04, 7.03, NFR-06. Local data: none beyond the request.

Algorithm 6 RESERVE(user, listingId, qty, idempotencyKey)

if an order with idempotencyKey exists **then return** that order
if user has ≥ 3 no-shows in the last 30 days **then raise** "reservations blocked" ▷ BR-06
if payment = cash **and** user is not trusted **then raise** "cash not allowed" ▷ BR-07
begin tx
n ← `UPDATE listing SET quantity = quantity − qty WHERE id = listingId AND quantity ≥ qty AND status = live`
if n = 0 **then rollback**; **raise** "sold out"
order ← insert Order(user, listing, qty, amount, code = UNIQUECODE(4 digits), status = reserved)
commit tx
publish "listing.quantityChanged" ▷ WebSocket clients update
Notifications.SEND(user, "reserved", order); Notifications.SEND(branch staff, "newOrder", order)
return order

5.7 Orders: Collect an Order

US-6.04, 7.03, 9.03.

Algorithm 7 COLLECT(staff, code)

order ← find order with pickupCode = code **and** status = reserved
if order not found **or** order.listing.branch $\notin$ staff.branches **then raise** "invalid code"
if now > order.listing.pickupEnd + config.lateMinutes **then raise** "window closed"
begin tx
order.status ← collected; order.collectedAt ← now
ImpactLedger.RECORD(orderCollected, order, {buyer, business})
commit tx
return "collected"

5.8 Impact Ledger: Record a Verified Action

US-9.01 to US-9.04, BR-09. Local data: the current impact factor per food category.

Algorithm 8 RECORD(sourceType, source, parties)

if a record with (sourceType, source.id) exists **then** ▷ idempotent: counted once
    return it
kg, egp, co2e, meals ← 0
for all line **in** source.items **do**
    f ← CURRENTFACTOR(line.category)
    w ← line.weightKg **or** line.quantity × catalogue.avgWeightKg
    kg ← kg + w; co2e ← co2e + w × f.kgCO2ePerKg; meals ← meals + w / f.kgPerMeal
    egp ← egp + (line.originalPrice − line.pricePaid) **if** sold, **else** w × f.avgPriceEgpPerKg
insert ImpactRecord(sourceType, source.id, parties, kg, egp, co2e, meals, f.version, now)

5.9 Donations: Route Unsold Food to an NGO

US-6.07, 8.02, 8.04, BR-08. Started by the scheduler at the end of each pickup window.

Algorithm 9 ROUTEUNSOLD(listing) and ONOFFEREXPIRED(offer)

left ← Marketplace.UNSOLDATWINDOWEND(listing)
if left = 0 **or not** listing.branch.autoDonate **then return**
ngos ← verified NGOs where listing.branch.location ∈ serviceArea **and** listing.category ∈ foodTypes **and** capacity left > 0 **and** open now
order ngos: business's defaultNgo first, then by distance (PostGIS)
OFFERNEXT(listing, ngos, tried = ∅)

procedure OFFERNEXT(listing, ngos, tried)
    next ← first NGO in ngos not in tried
    if next = none **or** now > listing.safeUntil **then** notify business "not collected"; **return**
    offer ← insert DonationOffer(next, listing, expiresAt = now + 30 min)
    Notifications.SEND(next coordinators, "donationOffer", offer)
    Scheduler.AT(offer.expiresAt, OnOfferExpired(offer))

procedure ONOFFEREXPIRED(offer)
    if offer.response = pending **or** declined **then** OFFERNEXT(offer.listing, ngos, tried ∪ {offer.ngo})

5.10 Notifications: Send a Notification

US-11.01, 11.02.

Algorithm 10 SEND(user, type, payload)

```
if type is switched off in user.notificationPrefs then return
n ← insert Notification(user, type, text in user.language, deepLink)
if now ∈ user.quietHours and type ≠ pickupReminder then
    Scheduler.AT(end of quiet hours, push(n))
else
    FCM.PUSH(user.deviceTokens, n)                    ▷ invalid tokens are removed
```

5.11 Mobile App: Synchronise Offline Changes

NFR-21, R-13. Local data: the sync queue of (entity, field, value, changedAt) entries in SQLite.

Algorithm 11 SYNC()

```
while connected and queue is not empty do
    batch ← next 50 entries of queue
    result ← API.POST(/pantry/sync, batch)
    for all entry in batch do
        if server.updatedAt(entry.field) > entry.changedAt then keep the server value    ▷ last write wins per field
        else server applies entry.value
    remove batch from queue; refresh local pantry from result
```

6. Human Interface Design

6.1 Overview of User Interface

Mobile app (households and deal seekers). The app opens on *Home*, which shows the items closest to expiry, today's deals nearby and the user's impact this week. A bottom bar gives five tabs:

- **Home**: at-risk items with one-tap *recipe* and *donate* actions; recipe suggestions (*Use it up*).
- **Pantry**: items grouped by fridge, freezer and cupboard and coloured by urgency. A large *Add food* button opens the capture methods: scan receipt, scan barcode, photo, voice and manual. After any AI capture the user sees a review screen and confirms with one tap. Once a week the pantry offers the 30-second swipe check.
- **Deals**: a map and a list of nearby surplus; a listing page with price, pickup window, allergens and *Reserve*; after reserving, a QR code and 4-digit code for pickup.
- **Donate**: choose sealed pantry items to donate, see partner drop-off points on a map, or buy a bag for an NGO.
- **Impact**: kilograms, pounds, CO_2e and meals saved; the monthly waste rate against the Egyptian average; *How we calculate*.

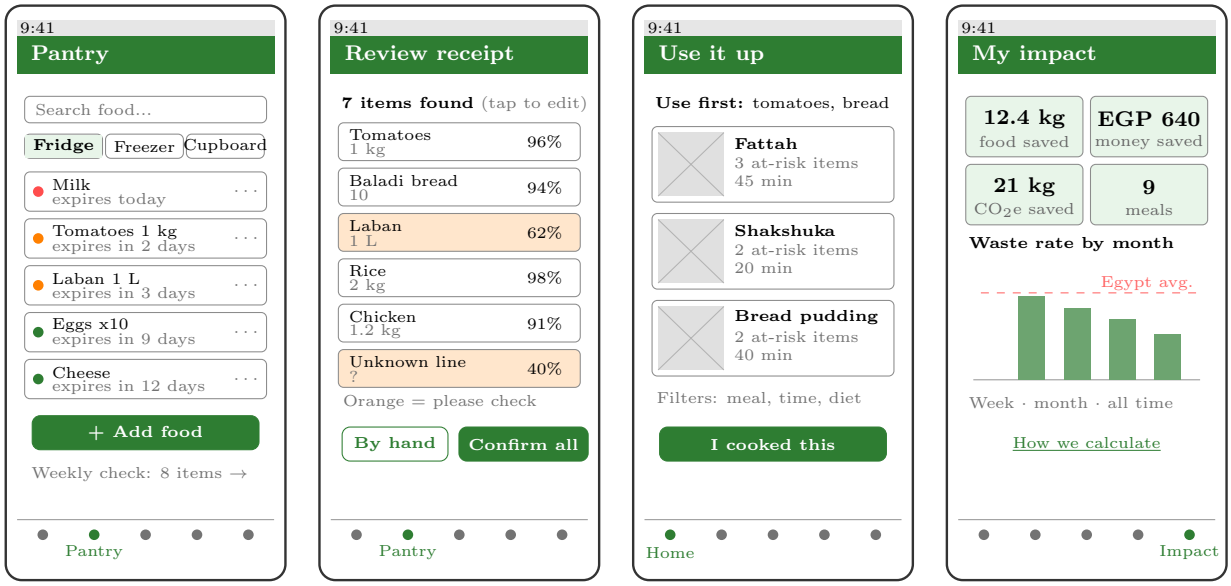
Web portal (businesses, NGOs, administrators, analysts). After sign-in the portal shows only the areas of the user's role in a left-hand menu. Businesses see *Today's orders* (with *Scan pickup code*), *Listings*, *Donations*, *Impact* and *Settings*. NGOs see *Offers*, *Pickups* and *Impact*. Administrators see *Verification*, *Moderation*, *Disputes*, *Settings* and *Analytics*. Analysts see *Exports*.

Feedback to the user. Every action gets visible feedback: a snackbar for success ("7 items added"), inline errors next to the field, a spinner with text for AI work longer than 1 s ("Reading your receipt…"), confidence percentages and orange highlighting on uncertain AI results, an offline banner, and an *Undo* for 10 s after automatic pantry changes. Push notifications open the related screen directly.

Language and accessibility. Arabic with a right-to-left (mirrored) layout is the default and English is available; the wireframes below are shown in English for readability. Every control has a text label and a screen-reader label, touch targets are at least 48 dp, and colour is never the only signal (SRS NFR-16 to NFR-18).

6.2 Screen Images

Figures 6.1 and 6.2 are low-fidelity wireframes of the main screens for the stories we implement during the course. They show layout and content, not final visual design. They will be replaced by screenshots of the real app as each sprint is delivered.



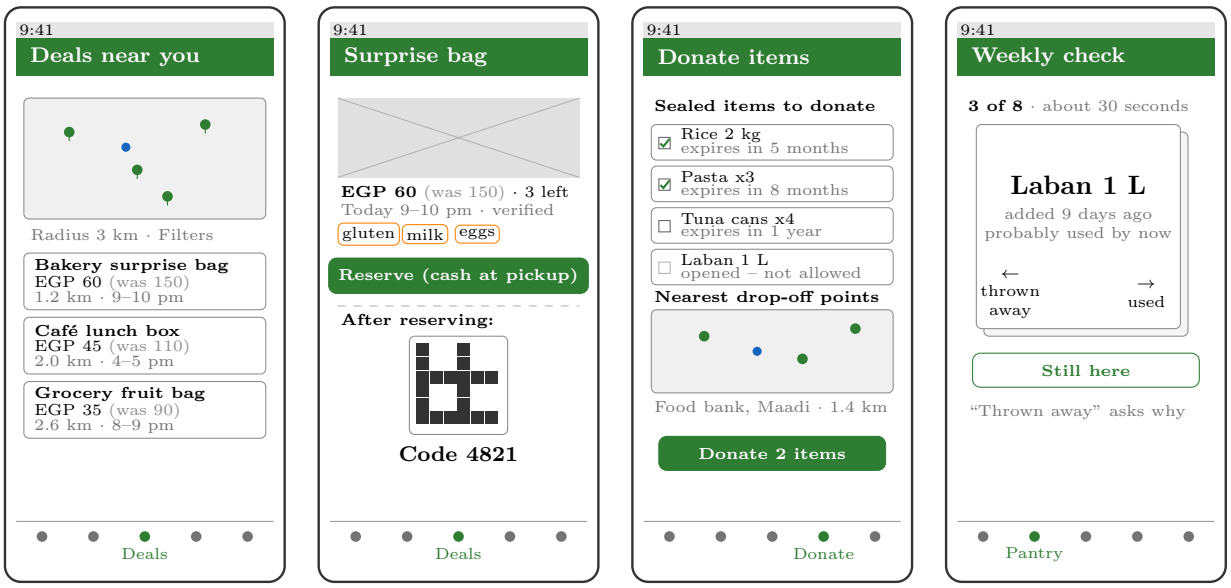
(a) Pantry (US-2.11) (b) Receipt review (US-2.02) (c) Recipe ideas (US-4.01) (d) Impact (US-9.01)

Figure 6.1: Wireframes of the household screens (mobile app).

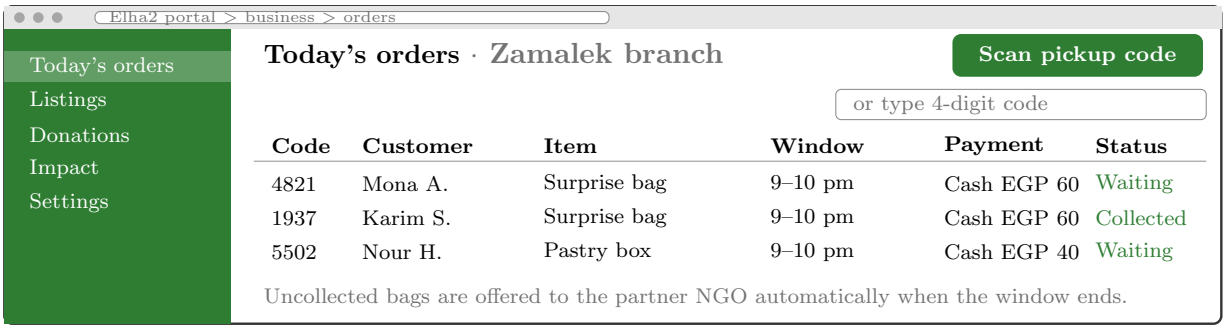
6.3 Screen Objects and Actions

Screen	Object	Action and system response	Stories
(a) Pantry	Storage tabs	Tap to filter by fridge, freezer or cupboard	US-2.11
	Item row with urgency dot	Red = expires today, orange = within 3 days, green = later. Tap opens the item with recipe, donate, move and “thrown away” actions	US-2.11, 2.10, 3.06
	Add food button	Opens the capture sheet: receipt, barcode, photo, voice, manual	US-2.01–2.07
	Weekly check link	Opens the swipe check (h)	US-2.09
(b) Review receipt	Item row with confidence	Tap to edit name, quantity, unit, storage or expiry; swipe to delete; orange = low confidence	US-2.02
	Confirm all	Saves every item to the pantry and shows “7 items added”	US-2.02
	By hand	Opens manual entry with autocomplete; used also when the AI fails	US-2.07
(c) Use it up	Recipe card	Tap to open steps scaled to the household	US-4.01

Screen	Object	Action and system response	Stories
	I cooked this	Deducts ingredients, adds impact, offers *Undo* for 10 s	US-4.05, 2.08
(d) My impact	Four totals	Tap a total to see it by Prevent, Sell and Donate; switch week, month, all time	US-9.01
	Waste-rate chart	Shows monthly rate against the Egyptian average line	US-9.04
	How we calculate	Opens the factors, sources and version dates	US-9.02
(e) Deals	Map pins and list cards	Tap a pin or card to open the listing; radius and filters at the top	US-5.01, 5.03
(f) Surprise bag	*Reserve*	Reserves the bag (cash at pickup) and shows the QR code and 4-digit code; "sold out" if the last bag was just taken	US-5.04, 7.02, 7.03
	Directions	Opens Google Maps to the store	US-5.07
(g) Donate items	Sealed-item checklist	Only sealed, unexpired pantry items can be ticked	US-8.06
	Drop-off map	Shows the nearest partner drop-off points and their opening hours	US-8.06
	Donate	Records the donation; the NGO confirms receipt, then the items leave the pantry and impact is added	US-8.06, 2.08
(h) Weekly check	Card stack	Swipe right = used, left = thrown away (asks for a reason), button = still here	US-2.09, 2.10
(i) Portal: today's orders	*Scan pickup code*	Opens the camera; a valid code marks the order collected and turns its status green	US-6.04
	Code field	Type the 4-digit code instead of scanning	US-6.04



(e) Deals (US-5.01) (f) Reserve and pickup (US-5.04) (g) Donate items (US-8.06) (h) Weekly check (US-2.09)



(i) Business portal: today's pickups (US-6.04), counter-staff view

Figure 6.2: Wireframes of the marketplace and donation screens (mobile app) and the business pickup screen (web portal).

7. Requirements Matrix

This chapter cross-references the design to the SRS. Traceability runs in both directions: every feature (E1 to E14), user story and non-functional requirement is realised by at least one component, and every component realises at least one requirement, so nothing is built without a requirement. Individual requirements trace on to their test cases through the SRS verification matrix (SRS Appendix D). When a requirement changes, these tables show which components to revisit; when a component changes, they show which requirements to re-verify.

7.1 Components to Requirements

Component	Functional requirements realised	Non-functional requirements
Mobile app	E1, E2, E3, E4, E5, E7, E8, E9, E10, E11	NFR-01, NFR-04, NFR-16 to NFR-18, NFR-21, NFR-27, NFR-28, NFR-30, NFR-31
Web portal	E6, E8, E9, E13	NFR-17, NFR-27, NFR-31
API layer	E1–E14 (authentication, validation, routing)	NFR-02, NFR-09, NFR-10, NFR-12, NFR-25
Accounts	E1, E10 (referrals)	NFR-10, NFR-13
Pantry	E2, E3 (digests), E4 (ingredient deduction)	NFR-04, NFR-21
Expiry	E3	–
Recipes	E4, E10 (Ramadan recipes, portion planner)	NFR-07
Marketplace	E5, E6, E12 (allergens, listing screening)	NFR-03, NFR-06
Orders	E7	NFR-06, NFR-11
Donations	E8 (including household donations)	NFR-19
Administration	E12, E13	NFR-08
AI gateway	E14; AI requirements in E2, E4, E6, E9	NFR-07, NFR-14, NFR-22, NFR-29, NFR-30
Impact ledger	E9, E10 (badges, streaks, challenges)	NFR-15
Notifications	E11, E10 (Ramadan prompts)	–
Scheduler	E3, E8 (timed jobs)	NFR-19
Integration adapters	E2 (barcodes, email), E5 (maps), E7 (payments)	NFR-11
ML service	E3 (shelf life), E5 (US-5.06)	NFR-29
PostgreSQL + PostGIS	All epics (persistence)	NFR-03, NFR-06, NFR-20
Deployment and operations (Appendix 8)	–	NFR-05, NFR-20, NFR-23, NFR-24, NFR-26

7.2 Functional Requirements to Components and Data

The table lists every functional requirement in the SRS by its identifier, the components that satisfy it (Chapter 3) and the main classes it reads or writes (Chapter 4). *Sprint* marks the stories we implement during the course (S1 to S3).

Req.	Title	Components	Main classes	Sprint
US-1.01	Sign up with a phone number	Mobile app; Accounts	User, Household	S1
US-1.02	Switch the app language	Mobile app; Accounts	User, Household	S1
US-1.03	Complete a short onboarding	Mobile app; Accounts	User, Household	
US-1.04	Set a location	Mobile app; Accounts; Adapters (Maps)	User	S1
US-1.05	Share a household pantry	Mobile app; Accounts	Household, User	
US-1.06	Register a business	Web portal; Accounts	Business, Branch	S1
US-1.07	Register an NGO	Web portal; Accounts	NGO	S1
US-1.08	Delete an account	Mobile app; Accounts; Scheduler	User	
US-2.01	Scan a receipt [**AI**]	Mobile app (OCR, masking); Pantry; AI gateway; Expiry	ReceiptDraft, PantryItem	S1
US-2.02	Review AI-extracted items [**AI**]	Mobile app; Pantry	ReceiptDraft, PantryItem	S1
US-2.03	Scan a barcode [**AI**]	Mobile app (ML Kit); Pantry; Adapters (Open Food Facts)	FoodCatalogueItem, PantryItem	
US-2.04	Import online grocery orders [**AI**]	Adapters (inbound email); Pantry; AI gateway	ReceiptDraft, PantryItem	
US-2.05	Photograph fridge or groceries [**AI**]	Mobile app; Pantry; AI gateway	ReceiptDraft, PantryItem	
US-2.06	Log items by voice [**AI**]	Mobile app; Pantry; AI gateway (speech, LLM)	ReceiptDraft, PantryItem	
US-2.07	Add an item manually	Mobile app; Pantry	PantryItem	S1
US-2.08	Update pantry automatically	Pantry; Recipes; Donations	PantryItem	S2
US-2.09	Complete the weekly check [**AI**]	Mobile app; Pantry; Scheduler	PantryItem	S2
US-2.10	Record thrown-away food	Mobile app; Pantry; Impact ledger	PantryItem	S1
US-2.11	View the pantry	Mobile app; Pantry	PantryItem	S1
US-3.01	Estimate an expiry date [**AI**]	Expiry; ML service	PantryItem, FoodCatalogueItem	S2
US-3.02	Read a printed date [**AI**]	Mobile app (OCR); Expiry; AI gateway	PantryItem	
US-3.03	Receive a daily expiry digest	Pantry; Scheduler; Notifications	PantryItem, Notification	S2
US-3.04	Explain date labels	Mobile app; Expiry	PantryItem	S2
US-3.05	Show at-risk items on the home screen	Mobile app (widget)	PantryItem	
US-3.06	Act on an expiring item	Mobile app; Notifications	PantryItem	
US-4.01	Suggest recipes for at-risk food [**AI**]	Mobile app; Recipes; Pantry; AI gateway	Recipe, PantryItem	S2
US-4.02	Ask the cooking assistant [**AI**]	Mobile app; Recipes; AI gateway	Recipe, PantryItem	
US-4.03	Filter recipes	Mobile app; Recipes	Recipe	
US-4.04	Scale a recipe [**AI**]	Recipes; AI gateway	Recipe	
US-4.05	Mark a meal as cooked	Mobile app; Recipes; Pantry; Impact ledger	PantryItem, ImpactRecord	S2

Req.	Title	Components	Main classes	Sprint
US-4.06	Save and share recipes	Mobile app; Recipes	Recipe	
US-4.07	Get leftover ideas [**AI**]	Recipes; AI gateway; Scheduler	Recipe	
US-5.01	Browse nearby surplus	Mobile app; Marketplace; PostgreSQL + PostGIS	Listing, Branch	S2
US-5.02	Filter listings	Mobile app; Marketplace	Listing	
US-5.03	View a listing	Mobile app; Marketplace	Listing	S2
US-5.04	Reserve a bag	Mobile app; Orders; Marketplace	Order, Listing	S2
US-5.05	Follow a store	Marketplace; Notifications	Listing, Notification	
US-5.06	See a personal feed [**AI**]	Marketplace; ML service	Listing, Order	
US-5.07	Get directions	Mobile app; Adapters (Maps)	Branch	S2
US-5.08	Rate a store	Mobile app; Orders	Order	
US-5.09	Report a problem with an order	Mobile app; Orders; Administration	Order	
US-6.01	Create a listing	Web portal; Marketplace	Listing	S2
US-6.02	Fill a listing from a photo [**AI**]	Web portal; Marketplace; AI gateway	Listing	
US-6.03	Manage staff roles	Web portal; Accounts	Organisation, Branch	
US-6.04	Confirm a pickup	Web portal; Orders; Impact ledger	Order, ImpactRecord	S3
US-6.05	Edit or cancel a listing	Web portal; Marketplace; Orders; Notifications	Listing, Order	
US-6.06	Import stock in bulk	Web portal; Marketplace	Listing	
US-6.07	Turn on auto-donation	Web portal; Accounts; Donations	Branch	S3
US-7.01	Pay in the app	Mobile app; Orders; Adapters (Paymob)	Order, Payment	
US-7.02	Pay cash at pickup	Orders	Order	S2
US-7.03	Show a pickup code	Mobile app; Orders; Impact ledger	Order, ImpactRecord	S2
US-7.04	Cancel a reservation	Mobile app; Orders; Marketplace; Scheduler	Order, Listing	
US-7.05	Refund a business cancellation	Orders; Adapters (payments); Notifications	Order, Payment	
US-7.06	View order history	Mobile app; Orders	Order	
US-7.07	Receive payouts	Web portal; Orders; Adapters (payments); Scheduler	Order, Payment	
US-8.01	Set donation preferences	Web portal; Accounts; Donations	NGO	S3
US-8.02	Route unsold food to an NGO	Donations; Scheduler; Notifications; Marketplace	DonationOffer, Listing	S3
US-8.03	Plan a collection route [**AI**]	Web portal; Donations; Adapters (Maps)	DonationOffer	
US-8.04	Accept a donation	Web portal; Donations	DonationOffer, NGO	S3
US-8.05	Donate a bag	Mobile app; Orders; Donations	Order, DonationOffer	
US-8.06	Donate pantry items	Mobile app; Donations	DonationOffer, PantryItem	
US-8.07	Record meals distributed	Web portal; Donations; Impact ledger	DonationOffer, ImpactRecord	
US-8.08	Issue a donation receipt	Web portal; Donations; Impact ledger	DonationOffer	
US-9.01	View personal impact	Mobile app; Impact ledger	ImpactRecord	S3
US-9.02	Explain the calculation	Mobile app; Impact ledger	ImpactFactor	S3
US-9.03	Count only verified actions	Impact ledger	ImpactRecord, ImpactFactor	S3

Req.	Title	Components	Main classes	Sprint
US-9.04	View the waste rate	Mobile app; Impact ledger; Pantry	ImpactRecord, PantryItem	S3
US-9.05	Get a monthly insight [**AI**]	Impact ledger; AI gateway; Scheduler	ImpactRecord, PantryItem	
US-9.06	Set a waste goal	Mobile app; Impact ledger; Notifications	ImpactRecord	
US-9.07	View the business dashboard	Web portal; Impact ledger	ImpactRecord, Listing	
US-9.08	Export a CSR report	Web portal; Impact ledger	ImpactRecord	
US-9.09	View the NGO dashboard	Web portal; Impact ledger	ImpactRecord, DonationOffer	
US-9.10	Export anonymised impact data	Web portal; Impact ledger	ImpactRecord	
US-9.11	Share an impact card	Mobile app; Impact ledger	ImpactRecord	
US-10.01	Earn badges and streaks	Mobile app; Impact ledger	ImpactRecord	
US-10.02	Join a neighbourhood challenge	Mobile app; Impact ledger	ImpactRecord	
US-10.03	Use Ramadan mode [**AI**]	Mobile app; Recipes; Notifications; Scheduler	Recipe	
US-10.04	Plan portions for guests [**AI**]	Mobile app; Recipes; AI gateway	Recipe	
US-10.05	Read storage tips	Mobile app; Notifications; Administration	Notification	
US-10.06	Invite friends	Mobile app; Accounts	User	
US-11.01	Receive push notifications	Mobile app; Notifications; Firebase (FCM)	Notification	S1
US-11.02	Set notification preferences	Mobile app; Notifications	User, Notification	
US-11.03	Get deal alerts	Notifications; Marketplace	Notification	
US-11.04	Get a pickup reminder	Notifications; Scheduler; Orders	Notification, Order	
US-11.05	View past notifications	Mobile app; Notifications	Notification	
US-12.01	Screen content automatically	Marketplace; Administration	Listing	
US-12.02	Report a user or content	Mobile app; Administration	Setting	
US-12.03	Show a verified badge	Accounts; Mobile app	Organisation	
US-12.04	Declare allergens	Mobile app; Marketplace	User, Listing	S3
US-13.01	Verify organisations	Web portal; Administration; Accounts	Organisation	S1
US-13.02	Moderate flagged content	Web portal; Administration	Listing	
US-13.03	Resolve disputes	Web portal; Administration; Adapters (payments)	Order, Payment	
US-13.04	Configure settings	Web portal; Administration	Setting	
US-13.05	View platform analytics	Web portal; Administration	Order, ImpactRecord	
US-13.06	Manage catalogue and recipes	Web portal; Administration; Expiry; Recipes	FoodCatalogueItem, Recipe	
US-14.01	Route all AI calls through a gateway	AI gateway	AIRequestLog	S1
US-14.02	Measure AI accuracy	AI gateway; CI pipeline	AIRequestLog	S2
US-14.03	Mask personal data	Mobile app (masking)	ReceiptDraft	S1
US-14.04	Log AI corrections	AI gateway	AIRequestLog	
US-14.05	Cap AI cost	AI gateway	AIRequestLog	

8. Appendices

Appendix A: Deployment Design

The backend and the ML service are stateless containers, so they can scale horizontally behind a load balancer (NFR-05). The hosting provider will be confirmed in Sprint 1 (SRS TBD-06); the table names the planned option for each part.

Node	Runs	Planned hosting
User phones	Mobile app	Google Play internal testing; Apple TestFlight
Browsers	Web portal (static build)	Firebase Hosting
API containers	Backend API with the Scheduler workers	Managed containers (e.g. Google Cloud Run)
ML containers	ML service; nightly training job	Same platform as the API
Database	PostgreSQL 16 with PostGIS	Managed PostgreSQL (e.g. Cloud SQL or Supabase), daily backups
Firebase	Authentication, FCM, Cloud Storage	Google Firebase

There are three environments: *local* (Docker Compose on each laptop), *staging* and *demo*. GitHub Actions runs linting, unit tests and the AI accuracy report (US-14.02) on every merge, and deploys to staging when the tests pass. Secrets such as API keys live in the platform's secret manager, never in the repository.

Appendix B: Design Change Procedure

The course requires both documents to be updated every sprint with tracking of changes [2]. At each sprint review we:

1. list the stories completed and any requirement that changed in the SRS, and use the requirements matrix (Chapter 7) to find the affected components;
2. update the affected diagrams, data dictionary entries, pseudocode and wireframes (replacing wireframes with screenshots of the delivered screens);
3. mark new or changed text with `\chg{version}{...}` (or `\chgt` inside tables) and removed text with `\del{version}{...}`, so the changes appear in blue and red with the version in the margin;
4. add a row to the revision history and tag the commit in GitHub (for example `docs-v1.1`), so every version of the SDD can be retrieved.

Appendix C: Declaration of AI Use

The design in this document was decided by our team, building on our Phase 0/1 work and the SRS. We used Claude (Anthropic) to help structure and draft this document, its diagrams and its LaTeX layout. All content was reviewed and edited by us, and we take full responsibility for the final document.